Does It Help When an LLM Chooses Where to Train? Opus-Chosen Layers for a LoRA Against Random and Rule-Based Choices

Juan José Giraldo Márquez
Independent researcher
github.com/JKUL-png

Juan José Gómez
Undergraduate student in Data Engineering and Artificial Intelligence
Universidad Autónoma de Occidente (UAO), Cali, Colombia

November 2026

Code and data: github.com/JKUL-png/llm-guided-lora

Abstract

Can a frontier language model that diagnoses a small model's failure choose better places to train it than chance? Qwen3.5-0.8B loops in thinking mode until it hits its token limit. We trained 24 LoRA adapters on the same 388 examples, each with exactly 2,654,208 parameters, and varied only which layers were trained: layers chosen by an informed Claude Opus session, random layers, a gradient-norm rule, a rule based on a layer sweep, Opus reading that sweep, and all 24 layers, with four seeds per arm. Every adapter escaped the loop: responses at the limit fell from 92.2% to 2.5–10.5%. On 100 fixed MATH-500 problems, Opus's layers scored −1.5 points against random layers [−8.8, +5.8] and +2.8 against the gradient rule [−5.2, +10.8], and our pre-specified criterion for a guided advantage was not met. No adapter beat simply turning thinking off on MATH. On GSM8K all adapters beat it by +10.0 to +12.8 points, but tied with a published weight edit of the same model; on IFEval they scored 6.0 to 12.5 points below that edit. Exploratory analyses suggest that the place does matter: the sweep rule beat the gradient rule by +7.2 points [0.0, +14.5], and the share of clean responses ranged from 57.0% to 81.2% across arms, lowest for Opus's layers. The place seems to matter, but Opus did not find a better one than chance.

1 Introduction

A natural use for a model that can read another model's internals is to decide where to intervene. A recent blog post by doubleAI places a coding agent inside training to find what a network fails to learn and to add code where it is needed (doubleAI, 2026). Work on weight-space programming has a frontier model read and edit the weights of small networks directly (Takehi et al., 2026). Both raise a question that is cheap to state and easy to get wrong: does the diagnosis help, compared with a choice that involves no diagnosis at all?

We study one concrete version of that question, on a model with a known, measurable failure. In thinking mode, Qwen3.5-0.8B often repeats itself until it reaches its token limit, so it rarely answers. In earlier work we showed that much of what a published weight edit gains on this model can be recovered by cheap baselines such as turning thinking off (Giraldo Márquez and Gómez,

2026). Here we keep the remedy fixed and vary only the place. The remedy is a LoRA adapter (Hu et al., 2022) trained to leave a loop and answer. The place is which four of the model's 24 layers the adapter trains. An informed Claude Opus session investigates the model with measuring tools and chooses four layers. We compare its choice with four layers drawn at random, four layers chosen by a gradient-norm rule, and two rules based on a systematic sweep in which each layer is switched off. Every adapter has exactly the same number of parameters and is trained on the same data, with the same steps and learning rate.

The design borrows the logic of a placebo. If the layers chosen by Opus help because Opus understood the network, they should beat random layers trained in exactly the same way. They should also beat a cheap automatic rule; otherwise the diagnosis adds cost and nothing else. We fixed the primary comparison and the criterion for a guided advantage before evaluation, in a plan kept in a local git repository. We call these *pre-specified*; the repository has no external timestamp, so we do not call them pre-registered. We say plainly which analyses were decided after seeing results (§3.7). An AI-assisted skeptical review of the analysis led us to adopt more conservative statistics, and we report them as primary.

Contributions.

1. A controlled test of LLM-guided layer selection: 6 arms, 24 adapters, equal parameters, data and training, four seeds, and a random-layer placebo (§3).
2. Two automatic arms that separate measuring from interpreting: a rule that reads a layer sweep, and Opus reading the same sweep (§3.3).
3. Evaluation on MATH-500, GSM8K and IFEval against two cheap references, turning thinking off and a published weight edit (`p50c`), with intervals that count the variation between adapters (§4).
4. An explicit account of what was fixed before and after seeing results, and of the corrections that followed an external-style review (§3.7).

Findings.

- **Opus did not beat random layers or the gradient rule.** The differences on MATH are -1.5 [-8.8, $+5.8$] and $+2.8$ [-5.2, $+10.8$] points, and the pre-specified criterion for a guided advantage was not met (§4.2).
- **Every adapter escapes the loop, but none beats turning thinking off on MATH** (§4.1, §4.3). On GSM8K every adapter does, and all tie with `p50c`; on IFEval they are below `p50c` (§4.4).
- **The place seems to matter** (exploratory). A sweep-based rule beats the gradient rule, and the share of clean responses varies widely across arms, with the layers chosen by Opus lowest (§4.5). The place seems to matter, but Opus did not find a better one than chance.

2 Background and related work

Weight-space programming and our earlier audit. Takehi et al. describe experiments in which a frontier LLM reads the weights of small networks and edits them by reasoning, without gradient descent (Takehi et al., 2026). In the best-known one, an edit to two tensors of Qwen3.5-0.8B raises its MATH-500 accuracy in thinking mode from a low base, because the unedited model mostly loops until it hits the token limit. In earlier work we rebuilt that edit (`p50c`: 130,188 changed entries in `mlp.down_proj` of layers 15 and 19) and showed that cheap baselines recover much of its gain (Giraldo Márquez and Gómez, 2026). A public audit of the original logs motivated several of those controls (Messing, 2026). We reuse that edit here only as a reference point, `p50c`, and its results come from the earlier study with the grading rule of this paper (§3.5). The unedited model's

loops are documented on its model card, which also notes that it runs in non-thinking mode by default (Qwen Team, 2026).

An agent that decides where to intervene (doubleAI). The question behind this paper comes from a blog post by doubleAI (doubleAI, 2026). In the post, a coding agent is placed inside training. It looks at how the network is learning, finds what it does not learn, and inserts small exact programs ("code layers") behind a gate that starts at zero, so that inserting them does not change the network; ordinary training then decides whether to open the gate. Two of their experiments do not change weights at all, but control sampling (a linear probe on activations that triggers re-sampling, and a Python policy written by the model). The post describes two controls that we borrow in spirit. A *twin* is, in their words, "an identical run from the same checkpoint on the same batches, with the new layer switched off". A *sham* "keeps the layer's shape and parameters but scrambles the information it carries". Our design differs in three ways. First, we do not insert code: we choose which layers of a LoRA adapter to train, so the agent's product is a place, not a program. Second, our placebo is not a sham. A sham keeps the place and damages the information; our placebo keeps the data and draws the place at random, so it tests whether *where* matters, not *what*. Our nearest equivalent to a twin is the same seed, the same data and the same steps with other layers, together with the untrained model. Third, for every arm we report the items that the post's text, as far as we could find, does not state for its experiments: the number of seeds, uncertainty intervals, sampling temperature and per-experiment sample sizes (a few sample sizes are given in places). We cite only what is visible in the post, and we do not claim that an item is absent from the experiments themselves, only that we did not find it in the text.

Edits against adapters trained on the same data. A figure from a web page on "weight-patching" (kevinpl07, 2026), which is all we saw of it, compares a weight edit with a LoRA trained on the same data on four bugs. Its title reads "Fine-tuning fixes the bug too. But it changes everything else." Its caption says that LoRA ties or wins on the bug in three of four cases, while the edit wins on side effects, and that a random edit of the same size does nothing. The suspicion is the same as ours: a sophisticated intervention should be set against a cheap baseline trained on the same data. The setting is different in every way that matters. They compare an edit with a LoRA; we compare ways of *choosing the layers* of a LoRA. Their figure states greedy decoding and shows no intervals or sample sizes, while we sample with temperature and four seeds. Their random edit is an edit, not a LoRA, so it is not our placebo. We therefore do not use it as evidence for or against our result.

Choosing where to adapt. Parameter-efficient fine-tuning with LoRA (Hu et al., 2022) leaves open which layers to adapt. Existing methods decide where to spend the parameter budget by importance scores computed during training (Zhang et al., 2023), or by sampling which layers to update according to their importance (Pan et al., 2024). Work on locating and editing facts points at MLP layers in the middle of the network (Meng et al., 2022) and describes feed-forward layers as key-value memories (Geva et al., 2021), which is part of why we restrict our adapters to the MLP. Directional ablation shows that a behavior can sometimes be removed by a rank-one edit (Arditi et al., 2024). None of this asks whether a language model, given a diagnosis task, picks better layers than chance, and that is the question here. Our gradient-norm arm is a simple member of the first family; our sweep arm is closer to ablation-style localization.

Handling loops and test-time control. Forcing a model to end its thinking is a simple way to control test-time compute (Muennighoff et al., 2025). We use the same idea as a *teacher*, not as an inference trick: the training examples are loops cut at the first copy and closed by hand (§3.2).

Benchmarks and tools. MATH-500 is the 500-problem subset of MATH (Hendrycks et al., 2021) used by Lightman et al. (2023). We also use GSM8K (Cobbe et al., 2021) and IFEval (Zhou et al., 2023). Generation uses vLLM (Kwon et al., 2023), and grading uses math-verify

(Kydlíček, 2026) and the lm-eval IFEval checker (Gao et al., 2023). For statistics we use Wilson intervals (Wilson, 1927), exact McNemar tests (McNemar, 1947), Holm's correction (Holm, 1979), the bootstrap (Efron, 1979) and a randomization test in the tradition of Fisher (1935).

3 Methods

3.1 Design

The question is whether a language model that diagnoses the problem and picks the layers to adapt does better than choosing the layers at random or by a cheap automatic rule. We train 24 LoRA adapters: 6 arms, each with 4 seeds. All arms train on the same data, in the same order for a given seed, for the same number of steps and with the same learning rate. All adapters have exactly 2,654,208 trainable parameters. Only the layers differ, and the rank that equalizes the parameter count. Table 1 lists the arms.

Table 1: The six arms. Layers are numbered 0 to 24−1. In arms (b), (c) and (f) the layers can change with the seed; in (d) and (e) they are the same in every seed. Source: `resultados/p3_lista_lora.csv`.

Arm	How layers are chosen	Layers trained, seeds 0 / 1 / 2 / 3
(a)	all 24 layers (rank 8)	all 24 in every seed
(b)	Opus, informed session	11, 15, 19, 22 / 14, 19, 22, 23 / 12, 15, 17, 20 / 10, 14, 15, 22
(c)	random placebo	1, 2, 4, 20 / 12, 14, 18, 21 / 7, 9, 10, 14 / 1, 5, 9, 15
(d)	gradient-norm rule	same in every seed: 10, 14, 15, 23
(e)	automatic sweep rule	same in every seed: 0, 10, 12, 15
(f)	Opus reading the sweep table	10, 14, 15, 17 / 10, 14, 15, 17 / 10, 14, 15, 18 / 10, 14, 15, 18

The contrasts that isolate the choice of layers are among the four-layer arms (b)–(f). Arm (a) is a reference: it differs from the rest in the layers *and* in the rank (8 against 48), so a difference with (a) cannot be attributed to either. The original model, with thinking and without thinking, and the `p50c` edit of earlier work, are fixed references in every table (§3.5). Turning thinking off is the cheap baseline that the earlier work showed to be strong (Giraldo Márquez and Gómez, 2026); it must therefore be in every comparison.

3.2 Model, adapters and training data

Model and adapter. All experiments use Qwen3.5-0.8B, which has 24 blocks, each with an MLP part. LoRA is applied to the MLP projections only. This choice makes the parameter count exactly comparable. If LoRA touched every module, a layer with linear attention and a layer with full attention would cost different numbers of parameters per rank, and two groups of layers could not be given the same budget. With MLP-only adapters every layer costs 13,824 parameters per unit of rank, so 24×8 and 4×48 both give 2,654,208 (checked with the formula and with the PEFT library). The scaling factor α is twice the rank (16 and 96).

Training data: cut and close. The model loops, so it almost never produces a finished, correct response that could serve as a training example. We therefore use the "cut and close" recipe of our earlier work. The unedited model is run on 1,200 problems from the MATH training set (disjoint from MATH-500; the data builder rejects any evaluation problem). When a response loops, a detector finds the first copy of the repeated passage. We cut the text there, append a fixed closing sentence and the end-of-thinking tag `</think>`, let the model write its answer, and keep the

example only if math-verify marks the answer correct. Responses that finish on their own and are correct are also kept. The result is 388 examples (303 forced and 85 natural; 6 dropped for length), with 3,269,949 tokens in total and a maximum length of 22,000 tokens. The teacher is the unedited model itself, not `p50c`, so that the MIT edit does not enter any arm. GSM8K and IFEval are never used for training; they measure transfer and forgetting.

Training. The loss counts only on the closing sentence and the answer. The earlier text is read but not trained on. This avoids reinforcing the path that led into the loop, and it makes memory use manageable, because the vocabulary has 248,320 tokens and full-sequence logits would not fit on the GPU. Each adapter is trained for 1 epoch (49 optimizer steps with gradient accumulation 8) at learning rate 2×10^{-4}. The learning rate was fixed in advance, and the pilot only checked that training did not diverge. Training runs under WSL2 on a single 8 GB consumer GPU. The seed fixes the order of the data and the sampling of evaluation, and in some arms also the layers (§3.3).

3.3 How the layers are chosen

(c) Random placebo. The layers are drawn as `random.Random(20261106+s).sample(range(24), 4)` for seed s. The draw was saved before any diagnosis and the trained layers match the saved file (checked by the macro script). Each seed has a different group of layers.

(d) Gradient-norm rule. In the unedited model, we compute for each layer the sum of the raw gradient norms of its three MLP matrices over 256 training examples drawn with the seed, and keep the four layers with the largest sum. The relative norm (divided by $\|W\|$) is recorded but not used.

(b) Opus, informed. A fresh Opus session investigates and then chooses four layers with reasons. The session runs outside this repository, in an isolated directory, and can only use a fixed set of tools: free commands to read the context and the help; short queries (activations, gradients, switching off a layer, a logit lens); and a command to generate samples (up to 32,768 tokens, several problems and samples per call, with the option of switching off the MLP of a layer). It sees only 40 problems from the MATH training set, never an evaluation problem. The budget is the same for every session: 50 short queries, 160 long samples and 150 calls. The context (about 650 words) contains the recipe, how the result will be measured, and summaries of the blog post of the original authors, of paper 1 and of the doubleAI post (only what is visible in it). It contains only aggregate numbers and no evaluation problem, and it does not contain the placebo draws, the layers of other arms, or the outputs of other sessions and of the pilot adapter. It does name layers 15 and 19, the layers that the MIT edit changes. This is an *anchoring* risk: Opus may choose them because it read them. We report it. Across the four sessions, layer 15 was chosen in 3 and layer 19 in 2, and two sessions share on average 1.33 of their four layers (0.67 would be expected by chance). The sessions are independent and have no memory. Each seed uses one session; the mapping is in Table 1 and in the list file. A rule fixed in advance decided whether two sessions were enough (if the first two shared at least three of their four layers) or four were needed. For (b) the first two sessions did not (any two sessions share at most 2 layers), so four were run.

Two asymmetries are part of the cost of "guided" and are reported: (b) and (f) receive a context and (for (b)) hours of GPU time through the tools, while (c), (d) and (e) do not.

(e) Automatic sweep. This arm separates *measuring* from *interpreting*. The system, not Opus, measures every layer. The sweep uses the 37 diagnostic responses that reach the token limit. In each one, the loop detector of paper 1 marks where the loop starts, and we take 8 fragments: the cut point at the first copy and 7 more at paragraph (or line) ends spread across the loop. That makes 296 fragments, plus 3 responses that do close on their own, used as a descriptive positive control. For each fragment we run forward passes (no generation) under 49 conditions: the normal

model and, for each of the 24 layers, its MLP output replaced by zero or by its mean. The main metric is the change in $\log P(\texttt{</think>})$ as the next token. Two secondary metrics are the change in the log-probability of the full closing sentence and of the next 32 real tokens. Per-layer means have bootstrap intervals over problems, because the fragments of one response are not independent. The rule, fixed before measuring, takes the four layers with the largest mean absolute change in the main metric under zeroing.

(f) Opus reading the sweep. Opus receives the same context as in (b) plus the sweep table, but no measuring tools and no GPU. It reads the table and chooses four layers with reasons. The same two-or-four rule applies; the first two sessions shared 3 of four layers, so two sessions were used, each for two seeds. Without (e), (f) could not be judged: if Opus picked the same four layers as the rule, its interpretation would add nothing.

3.4 Merging and size of the edit

Each adapter is merged into the base weights and the merged model is saved in bfloat16, so that vLLM evaluates a model with edited weights. Rounding to bfloat16 could erase a small update, so we measure the smallest fraction of the update that survives rounding (0.9968 over all tensors and adapters). We report, as descriptive numbers, the largest $\|\Delta W\|/\|W\|$ per tensor after merging for each arm: 0.0162–0.0164 for (a) and 0.0364–0.0452 across the four-layer arms. These are not tests, and arms with the same parameter count can still change the weights by different amounts.

3.5 Evaluation and grading

Problem sets. Evaluation uses the fixed subsets of paper 1, chosen once and never regenerated: 100 MATH-500 problems (stratified by difficulty level, with 8, 18, 21, 26, 27 problems at levels 1 to 5; selection seed 20261103), 100 GSM8K problems (the first of two subsets) and 100 IFEval prompts. No evaluation problem enters training or diagnosis. Every arm uses exactly these problems, with four sampling seeds ($100 \times 4 = 400$ (problem, seed) pairs per arm and task). With 100 problems, one run has a 95% interval of about ± 10 points near 50%, so we do not read small differences.

Generation. Evaluation uses vLLM under WSL2 on the merged models. The prompt is the problem, a newline, and "Please reason step by step, and put your final answer within \boxed{}." Table 2 gives the configurations. All LoRA arms use the thinking configuration of the original authors at $T = 0.6$, with presence penalty 1.5 and a limit of 32,768 tokens, and the seed of the LoRA is the sampling seed. The temperature $T = 1.0$ was measured only for the best arm at $T = 0.6$ (chosen by a rule fixed in advance: highest mean accuracy over four seeds, ties broken by fewer responses at the limit) and for the references. GSM8K and IFEval were measured only at $T = 0.6$ (§3.7).

Table 2: Generation configurations. Each configuration differs from the original thinking configuration only in the mode or the stated parameter. The third row is the no-thinking profile of the original authors; it appears in paper 1 and is listed here for completeness. Source: `scripts/comun_math.py`.

Configuration	**Thinking**	T	top-p	top-k	presence	repetition	max tokens
Thinking (original MIT; all LoRA arms)	on	0.6	0.95	20	1.5	1.0	32,768
No-thinking, same sampling (reference N)	off	0.6	0.95	20	1.5	1.0	32,768
No-thinking, MIT sampling (paper 1 only)	off	0.7	0.8	20	1.5	1.0	8,192

References. T: the unedited model with thinking. N: the unedited model with thinking off, same sampling and 32,768 tokens. P: `p50c`, the MIT edit, with thinking. They are in every

comparison. T and P come from the earlier study with the same fixed problems and four seeds, and were re-graded with the rule below.

Grading. The final answer is the text after the *last* `</think>`. In thinking mode, a response without `</think>` is wrong; without thinking there is no tag, and the whole text counts as the answer. For MATH and GSM8K we take the last `\boxed{}` and compare it with math-verify 0.9.0 (Kydlíček, 2026); no box is wrong. For IFEval we apply the official checker to that answer. The rule is the same for all arms and references. Our earlier study took the text after the *first* `</think>`. The rule used here was fixed before the real evaluation, but after we saw that a pilot LoRA sometimes closes its thinking more than once. To check that the choice does not matter, we graded all LoRA responses on MATH and GSM8K with both rules: they agree on 4,800 of 4,800 responses.

Metrics. For every arm we report accuracy, the share of responses that reach the token limit, mean tokens, the share of responses with more than one `</think>`, accuracy among the responses that finish, and the size of the merged edit. Two quantities are exploratory and were defined after seeing the data: a *clean response* (exactly one `</think>` and not at the limit), and the gain split between problems where the original thinking model hit the limit and those where it finished.

3.6 Statistics

The unit of analysis is the problem. The four seeds of the same problem are not independent, and neither are the four LoRAs of the same arm, which differ in layers, training seed and sampling at once. Our primary interval is therefore a **two-way bootstrap**: it resamples the problems and, separately, the seeds of each arm (4,000 resamples, fixed seed 20261110); it counts that a different LoRA would give a different number. With only four LoRAs per arm it is crude and probably somewhat narrow. As a robustness check we also report the bootstrap over problems alone (10,000 resamples, seed 20261107). Our primary test is an **exact sign-flip permutation test by problem**: for each problem we sum correct answers over the four seeds for arm X minus arm Y and compare the sum with its distribution when the sign of each problem's difference is flipped independently. p-values are corrected with Holm's method within the family stated in each table. The McNemar test on the 400 (problem, seed) pairs, which was our first choice, treats those pairs as independent. This pseudo-replication makes its p-values too small, so we keep it only as a robustness check. A comparison of four LoRAs against four LoRAs (each LoRA as one observation) is also given; its smallest possible p-value is 0.029.

Primary comparisons were fixed in advance: (b) against (c) and (b) against (d) on MATH at $T = 0.6$, with a Holm family of two, and every arm against the no-thinking original on each task. Everything else is exploratory. After the external review we adopted a reading rule for the results: a difference is *solid* only if the Holm-corrected permutation p-value is below 0.05 *and* the two-way interval excludes zero; if only one holds it is an *indication*. The rule was chosen after seeing the data. Because "did not beat" is not "is equal to", we report the intervals, which bound how large an advantage could remain hidden.

3.7 What was fixed before, and what after seeing results

"Pre-specified" here means written in the project plan before the results existed. It is not a pre-registration: the plan lives in a local git repository with no external timestamp, and its first commit is dated 2026-11-07. Table 3 says what was decided when.

The "guided wins" rule required (b) to beat (c) *and* (d) after Holm correction at both temperatures. Because of the budget decision, (b) was never evaluated at $T = 1.0$, so the rule's "partial evidence" branch cannot be judged.

Table 3: What was decided before and after seeing results. Dates are commit dates from the repository history.

Stage	Decisions
Before any evaluation	Primary test ((b) against (c) and (d), Holm family of two) and the "guided wins" rule; turning thinking off as a mandatory reference; the placebo draw; equal parameters in all arms; the rule for the best arm and the decision to measure $T = 1.0$ only there (commit dated 2026-11-07, before the first stage was run).
Before evaluation, after seeing something	The grading rule (after seeing multiple closes in a pilot adapter); arms (e) and (f), added once the layers of the first Opus sessions were known but before any training; the mapping of Opus sessions to seeds.
After seeing MATH	The Holm families (of two and of six) and the way seeds are pooled (analysis commit dated 2026-11-09, together with its outputs); the decision to measure GSM8K and IFEval only at $T = 0.6$ and to **drop the planned company-knowledge task** (commit dated 2026-11-09), on the grounds that the main question was already answered on MATH. The dropped task is the one where the choice of layers might have mattered most.
After the external review	The two-way bootstrap, the permutation test, the solid/indication rule, all comparisons against `p50c`, the clean-response metric, the symmetric noise floor and the alternative Holm families. They correct or extend the analysis, but they were chosen while looking at the data.

4 Results

All accuracies are percentages of the 400 (problem, seed) pairs of a model; differences are in percentage points. Unless stated otherwise, brackets are the two-way 95% interval, p is the exact permutation test by problem, and "Holm" is that p corrected within the family named in each table (§3.6). Every result is labeled *pre-specified* or *exploratory*, and *solid* or *indication* under the reading rule of §3.6. That rule, like several of the analyses below, was adopted after seeing the data (Table 3).

4.1 Every adapter escapes the loop

Table 4 and Figure 1 give the per-model results on MATH at $T = 0.6$. In thinking mode the original model answers 7.8% correctly, because 92.2% of its responses run into the 32,768-token limit. All 24 adapters escape this: their share of responses at the limit falls to 2.5–10.5%, and their accuracy is 45.5–58.5%. The difference from the original thinking model is +37.8 to +50.8 points for the six arms, and every two-way interval starts at or above +28.0 (exploratory, since the plan did not list this comparison; it holds on GSM8K and IFEval as well, Table 8). That the adapters beat a model that loops is expected. The questions of this paper are what the place of the adapter adds, and how the adapters compare with the cheap baselines.

Two features of Table 4 matter for everything that follows. First, retraining changes the result a lot. Arm (a) uses the same 24 layers in all four seeds, yet its seeds range from 40 to 53 correct answers, a spread of 13 points. This is why our primary interval counts the variation between the four adapters. Second, the reference `p50c` answers as many problems as the adapters (51.2%), but 29.2% of its responses still hit the limit, and it spends 2.8–5.1 times as many tokens per response as the adapters. When it does finish, it is more accurate than any of them (72.4% against 46.7–62.1%).

Table 4: MATH-500 (100 fixed problems, four seeds), $T = 0.6$, per model. Accuracy with its two-way 95% interval; "Per seed" is correct answers out of 100 for seeds 0–3; "At limit" is the share of responses that hit the 32,768-token limit; "Tokens" is the mean per response; ">1 close" is the share of responses that close their thinking more than once; the last column is the accuracy among responses that did not hit the limit. Source: `revision/rev_por_modelo.csv`.

Model	Accuracy % [95% CI]	Per seed	At limit %	Tokens	>1 close %	Acc. if finished %
(a) 24 layers	45.5 [35.5, 55.8]	53 / 40 / 49 / 40	2.5	4,179	17.8	46.7
(b) Opus, informed	54.0 [45.5, 63.0]	51 / 51 / 58 / 56	10.5	7,554	39.2	60.3
(c) Random placebo	55.5 [46.8, 64.2]	55 / 57 / 57 / 53	6.0	6,266	18.5	59.0
(d) Gradient rule	51.2 [42.5, 60.0]	51 / 54 / 54 / 46	9.5	7,344	29.5	56.6
(e) Sweep rule	58.5 [50.0, 67.2]	57 / 57 / 60 / 60	5.8	6,140	21.8	62.1
(f) Opus + sweep	52.0 [43.0, 61.0]	52 / 55 / 54 / 47	6.5	6,586	27.0	55.6
Original, no-thinking	51.8 [43.0, 61.0]	49 / 49 / 57 / 52	12.0	7,955	0.0	58.5
Original, thinking	7.8 [3.8, 12.5]	7 / 9 / 6 / 9	92.2	30,770	0.0	100.0
`p50c` (MIT edit), thinking	51.2 [42.8, 60.0]	51 / 51 / 50 / 53	29.2	21,436	10.5	72.4

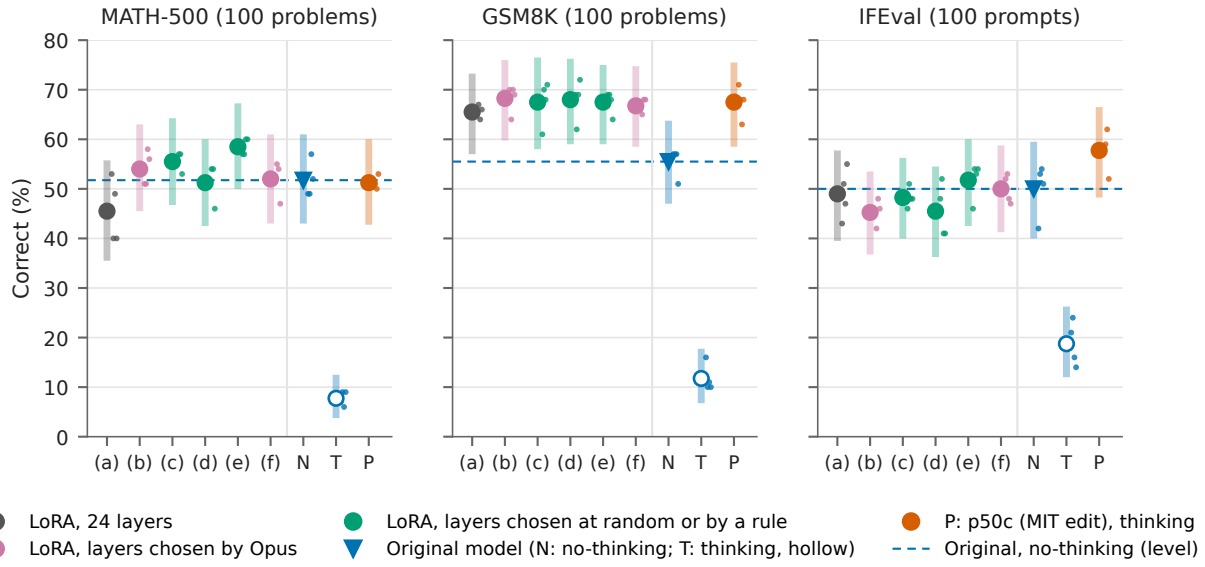


Figure 1: Accuracy per model at $T = 0.6$ on the three tasks. Large markers are the mean over four seeds, with the two-way 95% interval as a bar; small dots are the four seeds. The dashed line marks the original model without thinking (N). T is the original model with thinking, and P is `p50c`, the MIT edit, with thinking. Source: `revision/rev_por_modelo.csv`.

4.2 Primary comparison: Opus did not beat random layers or the gradient rule

Table 5 gives the pre-specified comparisons. The layers chosen by Opus (b) scored −1.5 points against random layers (c), with interval [−8.8, +5.8], and +2.8 against the gradient rule (d), with interval [−5.2, +10.8]. Neither is distinguishable from zero (Holm 0.69 for both). The robustness checks agree: McNemar on the 400 pairs gives 0.60 and 0.30, with 0.59–0.63 and 0.30–0.36 across all 24 ways of pairing the seeds of the two arms; treating each adapter as one observation gives 0.54 and 0.37; and the alternative grading rule gives identical grades (§3.5).

The "guided wins" criterion is not met. The pre-specified criterion required (b) to beat both (c) and (d) after Holm correction, at both temperatures. It fails at $T = 0.6$, and (b) was not evaluated at $T = 1.0$ (§3.7).

What the null bounds. "Did not beat" is not "is equal to". With four adapters per arm and 100 problems, the two-way intervals leave room for an advantage of Opus of up to +5.8 points over

random layers and +10.8 over the gradient rule. We did not fix a margin of equivalence in advance, so we do not run an equivalence test after the fact.

The secondary comparison, (b) against the 24-layer adapter (a), is +8.5 [−0.2, +17.8] (Holm 0.018 among all nine pre-specified MATH tests). It is an *indication*: the interval touches zero, and treating each adapter as one observation gives $p = 0.11$. Even if it held, it would not speak for Opus, because (a) differs from (b) in both layers and rank.

Table 5: Pre-specified comparisons on MATH at $T = 0.6$. Holm: family of two for the primary comparisons; family of all nine pre-specified MATH tests for (b)−(a). "Per-problem CI" and McNemar on the 400 pairs are robustness checks. Reading: solid / indication / – (§3.6). Source: `revision/rev_comparaciones.csv`, `revision/rev_holm.csv`.

Comparison	Diff. [95% CI]	p	p Holm	Per-problem CI	p McNemar	Reading
(b) − (c) random	−1.5 [−8.8, +5.8]	0.62	0.69	[−6.2, +3.2]	0.60	–
(b) − (d) gradient	+2.8 [−5.2, +10.8]	0.35	0.69	[−2.2, +8.0]	0.30	–
(b) − (a) 24 layers	+8.5 [−0.2, +17.8]	0.002	0.018	[+3.5, +13.8]	<0.001	indication

4.3 No adapter beats turning thinking off on MATH

The plan required every arm to be compared with the original model with thinking turned off, the cheap baseline that our earlier study found strong (Giraldo Márquez and Gómez, 2026). Table 6 gives these comparisons (all MATH differences are drawn together in Figure 2) and, as exploratory comparisons, those against `p50c`. Turning thinking off scores 51.8%, and **no arm beats it** under any of the Holm families we tried (Table 6; `revision/rev_holm.csv`). The closest is the sweep arm (e), at +6.8 [−0.5, +14.2], $p = 0.016$ before and 0.096 after correction. Training 2,654,208 parameters on 388 examples gives, on MATH, a result we cannot tell apart from changing one generation option.

Against `p50c`, only (e) is distinguishable from it, and it is above (+7.2 [+0.8, +14.5], Holm 0.047). By our reading rule this is solid, but it is exploratory and (e) was named the best arm using these same data, so it is inflated by selection (the winner's curse). Turning thinking off and `p50c` are tied: +0.5 [−7.2, +8.5].

Table 6: Each arm against the original model without thinking (pre-specified; Holm family of six) and against `p50c` (exploratory; Holm family of six). MATH, $T = 0.6$. Source: `revision/rev_comparaciones.csv`, `revision/rev_holm.csv`.

	Arm − original, no-thinking (pre-specified)				**Arm − `p50c`** (exploratory)			
Arm	Diff. [95% CI]	p	p Holm	Reading	Diff. [95% CI]	p	p Holm	Reading
(a)	−6.2 [−16.5, +3.8]	0.069	0.35	–	−5.8 [−14.5, +2.8]	0.037	0.19	–
(b)	+2.2 [−5.5, +10.2]	0.44	1.00	–	+2.8 [−4.8, +10.5]	0.36	1.00	–
(c)	+3.8 [−4.0, +11.2]	0.20	0.79	–	+4.2 [−2.5, +11.5]	0.15	0.61	–
(d)	−0.5 [−9.2, +7.5]	0.93	1.00	–	0.0 [−7.5, +7.8]	1.00	1.00	–
(e)	+6.8 [−0.5, +14.2]	0.016	0.096	–	+7.2 [+0.8, +14.5]	0.0079	0.047	solid
(f)	+0.2 [−8.0, +8.0]	1.00	1.00	–	+0.8 [−6.5, +8.0]	0.85	1.00	–

At $T = 1.0$. By the pre-specified budget rule, only the best arm at $T = 0.6$ (arm (e)) and the references were run at $T = 1.0$ (Table 7). This is the one measurement of (e) that is not inflated by its selection. It does not beat turning thinking off: +2.8 [−5.5, +10.8], $p = 0.43$ (pre-specified). Exploratory: at this temperature `p50c` loops again (64.2% at the limit), and (e) is +25.5 [+17.8, +33.5] points above it, as is the no-thinking original (+22.8 [+14.0, +31.8]).

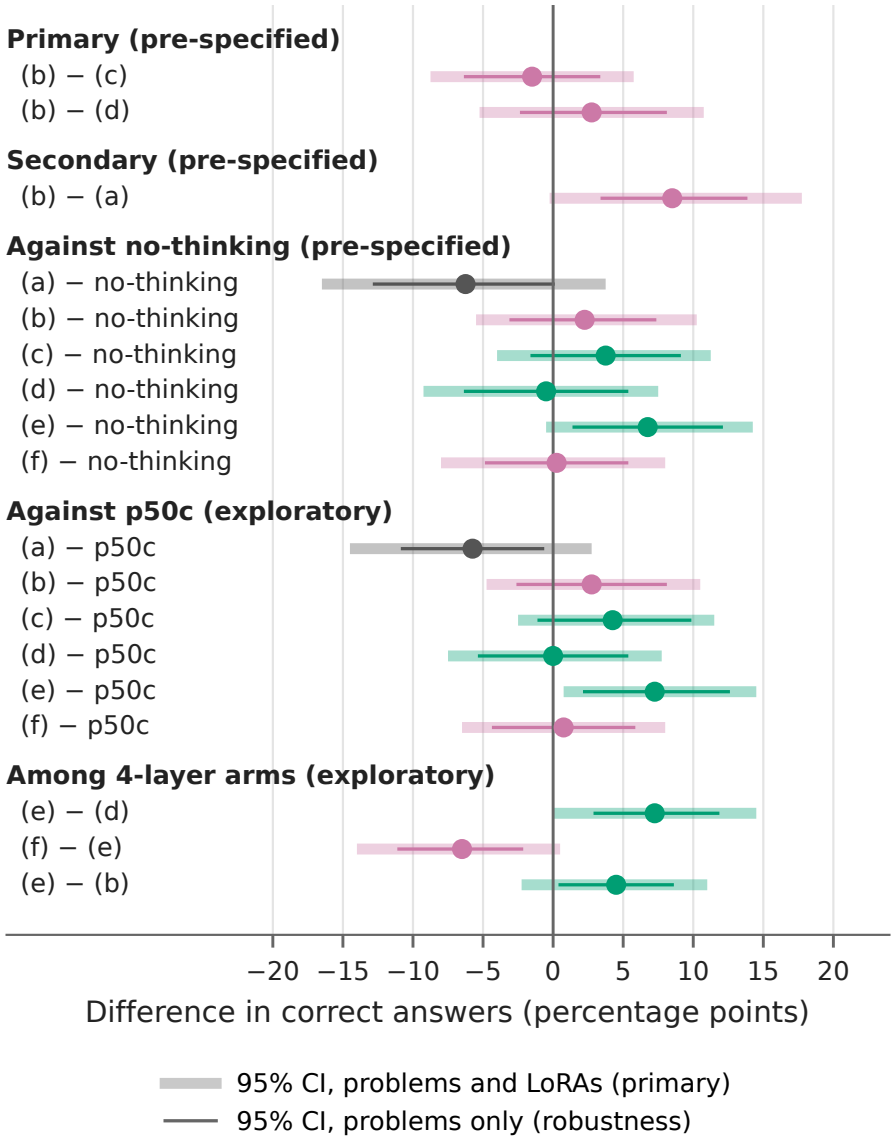


Figure 2: Differences in correct answers on MATH at $T = 0.6$. Wide light bars: two-way 95% interval (problems and adapters; primary). Thin dark bars: interval over problems only (robustness). Colors as in Figure 1. Source: `revision/rev_comparaciones.csv`.

Table 7: MATH at $T = 1.0$: the best arm at $T = 0.6$ and the references. Source: `revision/rev_por_modelo.csv`.

Model	Accuracy % [95% CI]	At limit %	Tokens
(e) Sweep rule	54.2 [46.0, 63.0]	0.2	5,539
Original, no-thinking	51.5 [43.5, 60.2]	1.0	8,064
Original, thinking	17.8 [11.0, 25.5]	82.2	29,662
`p50c` (MIT edit), thinking	28.8 [20.8, 37.0]	64.2	26,757

4.4 GSM8K and IFEval

GSM8K and IFEval were never used in training; they measure transfer and forgetting (Tables 8 and 9). Their comparisons against the no-thinking original are pre-specified; those against `p50c` are exploratory. Measuring them only at $T = 0.6$ was decided after seeing MATH (§3.7).

GSM8K: the adapters beat turning thinking off, and tie with `p50c`. All six arms are above the no-thinking original, by +10.0 to +12.8 points. This is solid for every arm (largest Holm <0.001) and survives a Holm family of all 18 arm-against-no-thinking tests across the three tasks. But the arms are tied with `p50c` (differences −2.0 to +0.8; smallest p 0.47), and the no-thinking original is below `p50c` by −12.0 [−19.8, −4.0]. On GSM8K, thinking helps whenever the model manages to finish, however the loop is fixed; this is not a particular merit of the adapters. Opus shows no advantage here either: (b)−(c) is +0.8 [−6.8, +8.8] and (b)−(d) is +0.2 [−7.2, +8.0] (exploratory).

IFEval: no detectable difference from turning thinking off, and below `p50c`. No arm differs from the no-thinking original (the smallest Holm p is 1.00). This is not "no harm". The intervals reach down to −14.8 points, the adapters reach the limit in 3.0–11.0% of responses (the no-thinking original in 0.0%), and they spend 6.8–16.8 times as many tokens. Against `p50c` all six arms are lower, by 6.0 to 12.5 points. By our reading rule this is solid for (b), (c) and (d), an indication for (a) and (f), and not significant for (e). An edit of 130,188 weights follows instructions better than an adapter of 2,654,208 parameters trained on math. The no-thinking original is also below `p50c`, by −7.8 [−18.0, +2.2]. Again Opus shows no advantage: (b)−(c) is −3.0 [−10.5, +4.2].

Table 8: GSM8K (100 problems) and IFEval (100 prompts), $T = 0.6$, per model. Same columns as Table 4. Source: `revision/rev_por_modelo.csv`.

	GSM8K			IFEval		
Model	**Accuracy % [95% CI]**	**At limit %**	**Tokens**	**Accuracy % [95% CI]**	**At limit %**	**Tokens**
(a) 24 layers	65.5 [57.0, 73.2]	1.8	2,327	49.0 [39.5, 57.8]	6.2	2,745
(b) Opus, informed	68.2 [59.8, 76.0]	4.5	3,420	45.2 [36.8, 53.5]	8.2	3,468
(c) Random placebo	67.5 [58.0, 76.5]	2.0	2,728	48.2 [40.0, 56.2]	7.8	3,416
(d) Gradient rule	68.0 [59.0, 76.2]	4.2	3,491	45.5 [36.2, 54.5]	11.0	4,389
(e) Sweep rule	67.5 [59.0, 75.0]	1.2	2,367	51.8 [42.5, 60.0]	4.0	2,191
(f) Opus + sweep	66.8 [58.5, 74.8]	1.8	2,514	50.0 [41.2, 58.8]	3.0	1,783
Original, no-thinking	55.5 [47.0, 63.8]	1.2	1,926	50.0 [40.0, 59.5]	0.0	261
Original, thinking	11.8 [6.8, 17.8]	87.8	29,304	18.8 [12.0, 26.2]	72.8	24,886
`p50c` (MIT edit), thinking	67.5 [58.5, 75.5]	6.0	11,627	57.8 [48.2, 66.5]	1.0	3,659

Table 9: GSM8K and IFEval: each arm against the no-thinking original (pre-specified) and against `p50c` (exploratory). Differences with two-way 95% intervals; Holm within the family of six arms per task and reference. †solid, *indication (§3.6). Source: `revision/rev_comparaciones.csv`, `revision/rev_holm.csv`.

	GSM8K				IFEval			
Arm	**− no-thinking**	**p Holm**	**− `p50c`**	**p Holm**	**− no-thinking**	**p Holm**	**− `p50c`**	**p Holm**
(a)	+10.0 [+2.5, +17.8]†	<0.001	−2.0 [−8.5, +4.8]	1.00	−1.0 [−11.0, +9.2]	1.00	−8.8 [−18.0, +0.8]*	0.014
(b)	+12.8 [+5.0, +21.0]†	<0.001	+0.8 [−5.8, +8.0]	1.00	−4.8 [−14.2, +5.0]	1.00	−12.5 [−21.8, −3.2]†	0.0028
(c)	+12.0 [+3.8, +20.5]†	<0.001	0.0 [−8.0, +7.5]	1.00	−1.8 [−11.5, +8.0]	1.00	−9.5 [−17.8, −1.5]†	0.0049
(d)	+12.5 [+5.0, +20.5]†	<0.001	+0.5 [−7.2, +8.0]	1.00	−4.5 [−14.8, +5.8]	1.00	−12.2 [−22.0, −3.0]†	<0.001
(e)	+12.0 [+5.0, +19.5]†	<0.001	0.0 [−6.2, +6.8]	1.00	+1.8 [−8.0, +11.8]	1.00	−6.0 [−15.0, +2.8]	0.068
(f)	+11.2 [+4.0, +19.0]†	<0.001	−0.8 [−7.0, +5.5]	1.00	0.0 [−9.0, +9.5]	1.00	−7.8 [−16.8, +1.0]*	0.043

4.5 Exploratory: the place seems to matter

Everything in this subsection is exploratory. It suggests that the choice of layers does matter, even though Opus did not find a better choice than chance.

Between automatic rules. The sweep rule (e) is +7.2 points above the gradient rule (d), with a two-way interval of [0.0, +14.5] and Holm 0.018 among the nine pairs of four-layer arms (Table 10). Both arms use the same layers in every seed, and the gap appears in each one (seeds 57 / 57 / 60 / 60 against 51 / 54 / 54 / 46). The interval touches zero, so it is an indication, not a proof. Opus reading the sweep table (f) scored −6.5 [−14.0, +0.5] against the automatic rule that read the same table ($p = 0.0074$ before correction; Holm 0.059 among the nine pairs, 0.037 in the family of five we first planned for these comparisons). This hints that the automatic rule outperformed Opus's reading of the same evidence, but it does not survive our stricter reading. Both comparisons involve (e), which was chosen as the best arm on these data (winner's curse), and which includes layer 0. Switching off layer 0 damages the model broadly (it ranks 2 of 24 layers in general damage, `barrido/dano_general_resumen.csv`), so its effect on closing the thinking may come from breaking the model rather than from being "the layer that closes".

Against the 24-layer adapter. The four-layer arms are above (a), solidly for random layers (c), at +10.0 [+1.2, +18.8], and for the sweep (e), at +13.0 [+4.8, +21.5]; as an indication for (b); and not significantly for (d) and (f) (Table 10). They also hit the limit more often than (a) (Table 4). Because (a) changes layers and rank at once, these comparisons do not say which of the two matters.

Table 10: Exploratory comparisons on MATH at $T = 0.6$ among arms that differ only in their layers, and of each four-layer arm against (a). Source: `revision/rev_comparaciones.csv`, `revision/rev_holm.csv`.

Comparison	Diff. [95% CI]	p	p Holm	Holm family	Reading
(e) − (d)	+7.2 [0.0, +14.5]	0.002	0.018	pairs of 4-layer arms (9)	indication
(f) − (e)	−6.5 [−14.0, +0.5]	0.0074	0.059	pairs of 4-layer arms (9)	–
(e) − (b)	+4.5 [−2.2, +11.0]	0.038	0.27	pairs of 4-layer arms (9)	–
(f) − (c)	−3.5 [−10.5, +3.8]	0.20	1.00	pairs of 4-layer arms (9)	–
(d) − (c)	−4.2 [−12.2, +3.5]	0.17	1.00	pairs of 4-layer arms (9)	–
(e) − (c)	+3.0 [−3.5, +9.5]	0.26	1.00	pairs of 4-layer arms (9)	–
(f) − (d)	+0.8 [−7.0, +8.5]	0.83	1.00	pairs of 4-layer arms (9)	–
(c) − (a)	+10.0 [+1.2, +18.8]	<0.001	0.0019	4-layer arm vs. (a) (5)	solid
(e) − (a)	+13.0 [+4.8, +21.5]	<0.001	<0.001	4-layer arm vs. (a) (5)	solid
(d) − (a)	+5.8 [−3.5, +14.8]	0.03	0.06	4-layer arm vs. (a) (5)	–
(f) − (a)	+6.5 [−2.8, +16.0]	0.031	0.06	4-layer arm vs. (a) (5)	–
(b) − (a)	+8.5 [−0.2, +17.8]	0.002	0.0059	4-layer arm vs. (a) (5)	indication

Clean responses. Accuracy and the share at the limit miss a residual failure: an adapter can close its thinking, keep reasoning where the answer should be, and close again. We call a response *clean* when it closes its thinking exactly once and does not hit the limit (a metric defined after seeing the data). The share of clean responses varies with the place much more than accuracy does (Table 11, Figure 3): from 57.0% to 81.2% across arms on MATH. On this measure the layers chosen by Opus are the worst. They are below random layers by −20.8 points [−38.2, −1.8], below the sweep by −17.2 [−27.8, −7.0] and below the 24-layer adapter by −24.2 [−38.2, −9.0]. Against the gradient rule the gap is −9.0 [−20.8, +2.8], and against `p50c` it is −4.0 [−14.5, +5.8]. The place also matters inside the placebo arm: its four random groups of layers give 75 / 53 / 92 / 91 clean responses out of 100 (layers 1, 2, 4, 20; 12, 14, 18, 21; 7, 9, 10, 14; 1, 5, 9, 15), which is why its interval is so wide. Responses that close more than once are right only 22.5–29.9% of the time, against 51.1–75.4% for clean ones, so the share at the limit captures only part of the residual

problem. On GSM8K and IFEval the clean shares are higher (82.5–95.2% and 88.8–96.8%).

Table 11: Exploratory: clean responses (exactly one close of thinking, not at the limit) on MATH at $T = 0.6$. Two-way 95% intervals. Source: `revision/rev_respuestas_limpias.csv`.

Model	Clean % [95% CI]	Per seed	Acc. among clean %	Acc. among >1 close %
(a) 24 layers	81.2 [69.5, 91.0]	65 / 84 / 85 / 91	51.1	22.5
(b) Opus, informed	57.0 [46.5, 67.2]	60 / 45 / 61 / 62	75.4	28.0
(c) Random placebo	77.8 [59.8, 92.5]	75 / 53 / 92 / 91	64.3	29.7
(d) Gradient rule	66.0 [56.2, 76.0]	58 / 61 / 69 / 76	67.0	23.7
(e) Sweep rule	74.2 [66.8, 81.8]	71 / 72 / 80 / 74	70.0	29.9
(f) Opus + sweep	69.0 [60.5, 77.2]	64 / 65 / 75 / 72	65.6	25.0
`p50c` (MIT edit), thinking	61.0 [53.0, 69.0]	57 / 62 / 62 / 63	76.6	42.9

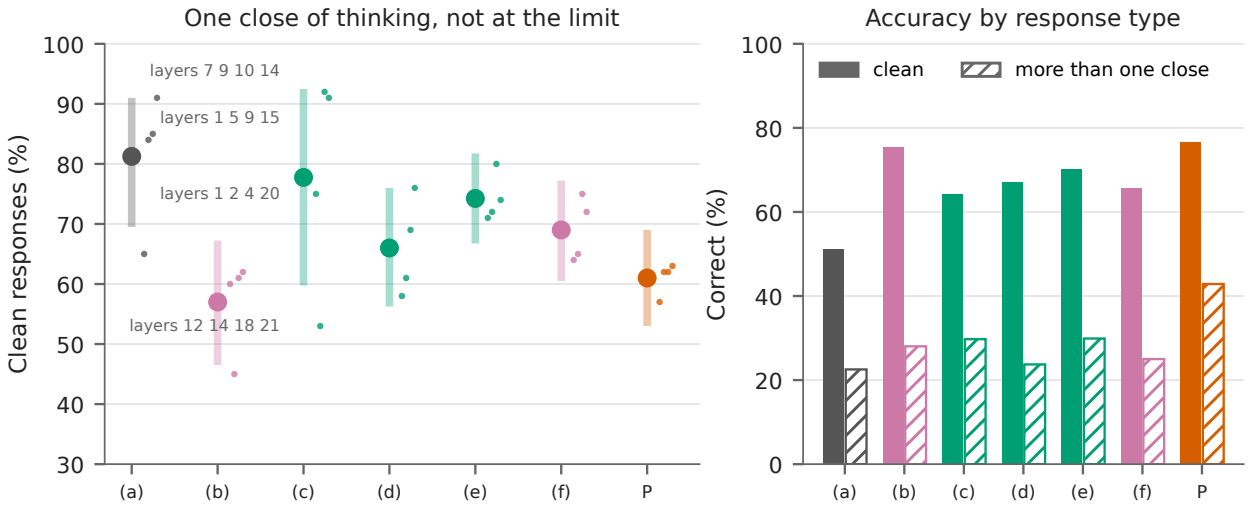


Figure 3: Exploratory. Left: share of clean responses on MATH ($T = 0.6$), with two-way 95% intervals and the four seeds; the placebo's four random groups of layers are labeled. Right: accuracy among clean responses (filled) and among responses that close their thinking more than once (hatched). Source: `revision/rev_respuestas_limpias.csv`, `p3_lista_lora.csv`.

4.6 Other exploratory analyses

Where the gain comes from. The plan asked for the gain to be split by whether the original thinking model hit the limit (Table 12). On MATH, the original hit the limit in 369 (problem, seed) pairs and finished in 31, where it was right 100.0% of the time. Almost all of the gain comes from the first group, as expected. In that group the arms score 41.2–55.0%, around the no-thinking original (48.8%) and `p50c` (47.2%).

Multiple closes. Responses that close their thinking more than once are much less accurate (on MATH, 26.5% against 62.5% for a single close, over the 24 adapters). But within the same problem, comparing seeds that closed once with seeds that closed more than once, the difference nearly disappears: +3.1 points [−1.7, +8.2] over 302 problems, with 71 won by each side and 160 ties (intervals over problems; `p3/cierres_por_problema_todos_resumen.csv`). Multiple closes appear mostly on hard problems, so the comparison across seeds mixes difficulty with the effect of the extra close. It does not show that extra closes cost nothing. Grading the first answer segment instead of the last would credit 0 of the 615 multi-close MATH responses, against 163 with our rule

Table 12: Exploratory: accuracy (%) on MATH at $T = 0.6$ split by whether the original thinking model hit the limit on the same (problem, seed) pair. Source: `revision/rev_ganancia_separada.csv`.

Model	Where the original looped	Where the original finished
(a) 24 layers	41.2	96.8
(b) Opus, informed	50.7	93.6
(c) Random placebo	52.0	96.8
(d) Gradient rule	47.7	93.6
(e) Sweep rule	55.0	100.0
(f) Opus + sweep	48.2	96.8
Original, no-thinking	48.8	87.1
`p50c` (MIT edit), thinking	47.2	100.0

(`revision/rev_regla_cierre_items.csv`).

Noise floor. Two seeds of the same model already disagree on many problems: for the no-thinking original, on 24.8% of (problem, seed) pairs on MATH, 26.7% on GSM8K and 20.7% on IFEval. We compare how often an arm and the original disagree with the average of the two models' own floors (`revision/rev_ruido_simetrico.csv`). On MATH, only (a) changes answers more than chance does (+5.0 [+1.3, +9.2]). On IFEval every arm does (+5.3 to +7.9), but so does `p50c` (+6.4 [+2.2, +11.2]). This is what happens when a model that thinks is compared with one that does not, not a signature of the adapters.

Problems that improve and worsen. Against the no-thinking original, per (problem, seed) pair, the arms worsen 8.8–17.5% and improve 11.2–16.0% of pairs on MATH (net −6.2 to +6.8). On GSM8K they worsen 5.8–8.8% and improve 18.2–20.5%; on IFEval they worsen 14.2–17.8% and improve 12.8–16.8% (`p3/cambios_*.csv`). Against the original *with* thinking, almost nothing worsens (GSM8K 0.2–0.8%).

4.7 Cost of choosing the layers

Training and evaluation cost the same in every arm: 12.9–61.0 GPU minutes to train one adapter and 13.7–22.5 minutes to evaluate it on the three tasks at $T = 0.6$. Arms differ in what it costs to choose the layers (`costo_diagnostico.csv`). The 24-layer adapter and the placebo cost nothing. The gradient rule took 37.6 GPU minutes for its four seeds. The sweep took 126.3 GPU minutes, plus 22.3 minutes to generate the stored diagnostic responses. The 4 informed Opus sessions used up to 240.1 GPU minutes through their tools (an upper bound that includes a 50.1-minute hung generation) and an API-equivalent $2.55 reported by the client; the 2 sessions that read the sweep cost $0.47 and no GPU. The most expensive way of choosing layers did not do better than the free ones.

5 Discussion

What the null result says. In this setting, an informed Opus session that investigates the model and chooses four layers for a LoRA adapter did not beat four random layers or a gradient-norm rule. This holds under every test we tried: the pre-specified McNemar test, the permutation test, treating each adapter as one observation, all pairings of seeds, and both grading rules. It is a statement about this protocol, not about language models choosing where to train in general. The intervals leave room for a modest advantage that our four adapters per arm could not detect (§4.2). The evidence does not, however, point towards one: on clean responses, the layers chosen by Opus are the worst of all arms.

The place seems to matter. The null result is not that "any layers will do". The automatic sweep beat the gradient rule by +7.2 points, and the share of clean responses ranged widely across arms and even across the four random draws of the placebo. Both signals are exploratory and both involve arms chosen or defined after seeing data, so we read them as indications. Taken together they suggest that where the adapter goes changes what it learns, and that Opus did not find a better place than chance. A single sentence summarizes it: *the place seems to matter, but Opus did not find a better one than chance.*

Why might Opus not help? Our data suggest three reasons, none of them tested directly. First, its evidence was thin: the whole budget of a session was 160 long samples, which is enough to see large effects but not differences of a few points. Second, its choices were unstable: two sessions share on average 1.33 of four layers, little more than the 0.67 expected by chance. Third, its context named layers 15 and 19, and it chose layer 15 in 3 of four sessions and layer 19 in 2, so its choices may partly reflect the hint rather than its own measurements. When the measuring was taken away from Opus and done by a fixed sweep, a mechanical reading of the sweep did at least as well as Opus reading the same table (§4.5). That points at interpretation, not only measurement, as a weak link, but the comparison is exploratory.

Against the cheap baselines. Our earlier study found that turning thinking off recovers most of what a weight edit buys on this model (Giraldo Márquez and Gómez, 2026). The same holds for trained adapters. On MATH no adapter beats turning thinking off. On GSM8K every adapter does, but so does `p50c`, and the adapters are tied with it. On IFEval the adapters are no better than turning thinking off and are below `p50c`. What the adapters buy over the edit is mostly efficiency on math: they finish far more often and with far fewer tokens, and the one adapter we ran at $T = 1.0$ did not collapse there as `p50c` does.

Relation to other work. doubleAI asks whether an agent that diagnoses a network can decide where to intervene during training (doubleAI, 2026). We test a narrow version of that question, with a placebo that keeps the data and draws the place, and the answer here is no advantage. Our placebo is not their sham, and our adapters are not their code layers, so this does not contradict their results; it shows that the question needs controls of this kind. The weight-patching figure (kevinpl07, 2026) reports that a LoRA trained on the same data ties or beats an edit on most bugs while changing more of everything else. We see a related pattern from the other side: the adapters match `p50c` on math but fall behind it on instruction following. We do not use either source as evidence for our result.

6 Limitations

- **Few adapters.** Each arm has four adapters, and each group of layers was trained once. The seed fixes the training order, the sampling and, in arms (b), (c) and (f), the layers, so these sources of variation cannot be separated. The two-way bootstrap tries to account for them, but with four units it is crude and probably somewhat narrow.
- **Analysis choices made after seeing data.** The Holm families, the pooling of seeds, the reading rule, the two-way interval, the permutation test and every comparison with `p50c` were chosen after seeing results (Table 3). We report alternative families in the repository (`revision/rev_holm.csv`); the main conclusion does not change under any of them.
- **Pre-specified, not pre-registered.** The plan lives in a local git repository without an external timestamp.
- **Reduced scope.** The planned company-knowledge task was dropped after seeing the MATH results, and it is the task where the place of the adapter might have mattered most. Arm (b)

was never evaluated at $T = 1.0$, so the "guided wins" criterion could only fail.

- **No control for the data.** All 24 adapters use the same 388 examples. No arm uses the same layers with other data, and there is no sham arm, so nothing here says how much of escaping the loop is due to the data recipe.
- **Opus with a hint.** Arms (b) and (f) are Opus with context that names the layers of the MIT edit. No un-hinted Opus arm was trained, and the four informed sessions agree little with each other. Arms (b) and (f) also had information and compute that (c), (d) and (e) did not.
- **Arm (a) is not a clean contrast.** It differs from the others in layers and rank at once.
- **The sweep arm.** Arm (e) includes layer 0, whose removal damages the model broadly, and it was chosen as the best arm with the same data, so comparisons with it are inflated.
- **References from another study.** The thinking original and `p50c` come from our earlier study (Giraldo Márquez and Gómez, 2026): same fixed problems, same sampling and four seeds, re-graded with this paper's rule.
- **Small evaluation sets and one model.** 100 problems per task, one model of 0.8B parameters and one training recipe. With these sets, differences of a few points are below what we can detect.

7 Conclusion

We asked whether a frontier model that diagnoses a small reasoning model's loops chooses better layers for a LoRA adapter than chance or a cheap rule. It did not: with equal parameters, data and training, Opus's layers did no better than random layers or a gradient-norm rule, and the pre-specified criterion for "guided wins" was not met. Every adapter escaped the loop, but on MATH none beat simply turning thinking off. Exploratory results suggest that the choice of layers does change what the adapter learns. In short, the place seems to matter, but Opus did not find a better one than chance. Settling this would need more random draws and retrainings of the same layers, a fake-data arm, and the guided arm at both temperatures; we have not run them.

8 AI use statement

AI is part of the method of this paper. Arms (b) and (f) are layer choices made by Claude Opus sessions run in isolation (§3.3). The code, the orchestration of the experiments, the analysis scripts, the scripts that generate every number, table and figure of this paper, and the drafts of the text were written with Claude Code (Anthropic), using Claude Sonnet 5.5 and Claude Opus 5.5. A separate Claude Fable 5.2 session, started without the project's conversation history and instructed to act as a skeptical external reviewer, audited the analysis before this paper was written; its report and the analyses it added are in the repository (`revision/REVISION_EXTERNA.md`). It was not independent of the project's tooling, and all its corrections were adopted. Every number in the paper is produced by a script from the logged CSV files, and the scripts check that the statements made in words match the data. The authors read and revised the final text and take full responsibility for its content.

Author contributions

Juan José Giraldo Márquez conceived and designed the study, ran all the experiments, carried out the analysis and wrote the paper with AI assistance. Juan José Gómez produced the figures,

with AI assistance (Claude Fable 5.2) from the logged data, and reviewed and approved the final manuscript.

Acknowledgments

We thank the authors of the weight-space programming work for publishing their edit, code and logs, and Sol Messing for the public audit that shaped the controls of our earlier study. No external funding; all experiments ran on a single consumer GPU.

References

Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL `https://arxiv.org/abs/2406.11717`.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021. URL `https://arxiv.org/abs/2110.14168`.

doubleAI. Can LLMs redesign themselves as they train? Blog post, 6 November 2026, 2026. URL `https://www.doubleai.com/research/can-llms-redesign-themselves-as-they-train`.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

Ronald A. Fisher. *The Design of Experiments*. Oliver and Boyd, 1935.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation. Zenodo, v0.4.0, 2023.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

Juan José Giraldo Márquez and Juan José Gómez. Decomposing the gain: How much of a weight-space edit's fix for reasoning loops do cheap baselines recover? Preprint, November 2026, 2026. URL `https://github.com/JKUL-png/decomposing-the-gain`.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021. URL `https://arxiv.org/abs/2103.03874`.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

kevinpl07. Weight patching. Web page; we only saw one figure from it, 2026. URL https://kevinpl07.github.io/weight-patching.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

Hynek Kydlíček. Math-Verify: a library to rule-based verify mathematical answers. Hugging Face, version 0.9.0, 2026. URL https://github.com/huggingface/math-verify.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. arXiv preprint arXiv:2305.20050, 2023. URL https://arxiv.org/abs/2305.20050.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

Sol Messing. Public audit thread on X of the weight-space programming logs. X, November 2026, 2026. URL https://x.com/solomonmg/status/2117650796342894884.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. URL https://arxiv.org/abs/2501.19393.

Rui Pan, Xiang Liu, Shizhe Diao, Renjie Pi, Jipeng Zhang, Chi Han, and Tong Zhang. LISA: Layerwise importance sampling for memory-efficient large language model fine-tuning. In *Advances in Neural Information Processing Systems*, 2024.

Qwen Team. Qwen3.5: Towards native multimodal agents. February 2026. Model card: https://huggingface.co/Qwen/Qwen3.5-0.8B, 2026. URL https://qwen.ai/blog?id=qwen3.5.

Rikiya Takehi, Jayden Teoh, Alex Zhang, and Omar Khattab. Sparks of weight-space programming: LLMs can natively read & write neural network weights. Blog post, 2 November 2026, 2026. URL https://rikiyat.github.io/blog/weight-space-programming/.

Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023. URL https://arxiv.org/abs/2311.07911.